

Azure Landing Zones in der Praxis - Was funktioniert, was nicht?

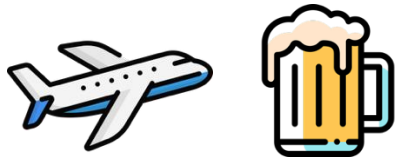
Julian Mooren



About Me



Julian Mooren
Azure Cloud Architect



15+ Jahre Erfahrung im Microsoft-Umfeld

Azure Plattform

Landing Zones · Core Infrastructure · Governance

Identity

Entra ID · Access Management · Hybrid Identity

Automation

Infrastructure as Code · DevOps

End User Computing

Azure Virtual Desktop · Virtual Desktop Infrastructure

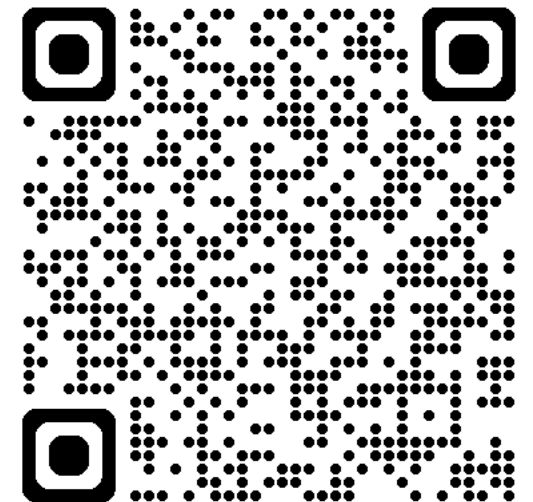
Application Delivery

Load Balancing · Reverse Proxy · Traffic Management

Cloud Plattform Community (Stuttgart)



- **15.Oktober 2026**
- **Collab: Azure Meetup Stuttgart**
- **Session 1: StackIT & Kubernetes**
- **Session 2: Azure IoT**



Aus dem echten Leben...

Habt ihr
Anforderungen und
Zielbild definiert?

Okay, dann machen
wir erstmal schnell...

Wir brauchen eine
Subscription!

Machen wir später.
Ist nur ein kleiner
Workload

3 Monate später???

Die unbequeme Wahrheit

“Nicht Azure bremst , sondern die Plattform darunter.”

Landing Zone ist das Fundament für alle Workloads.

- Schlechte Struktur gewählt → keine Skalierungsmöglichkeiten
- Nachträgliche Governance → sehr aufwendig
- Netzwerkfehler → bleiben jahrelang (Never touch! 💣)

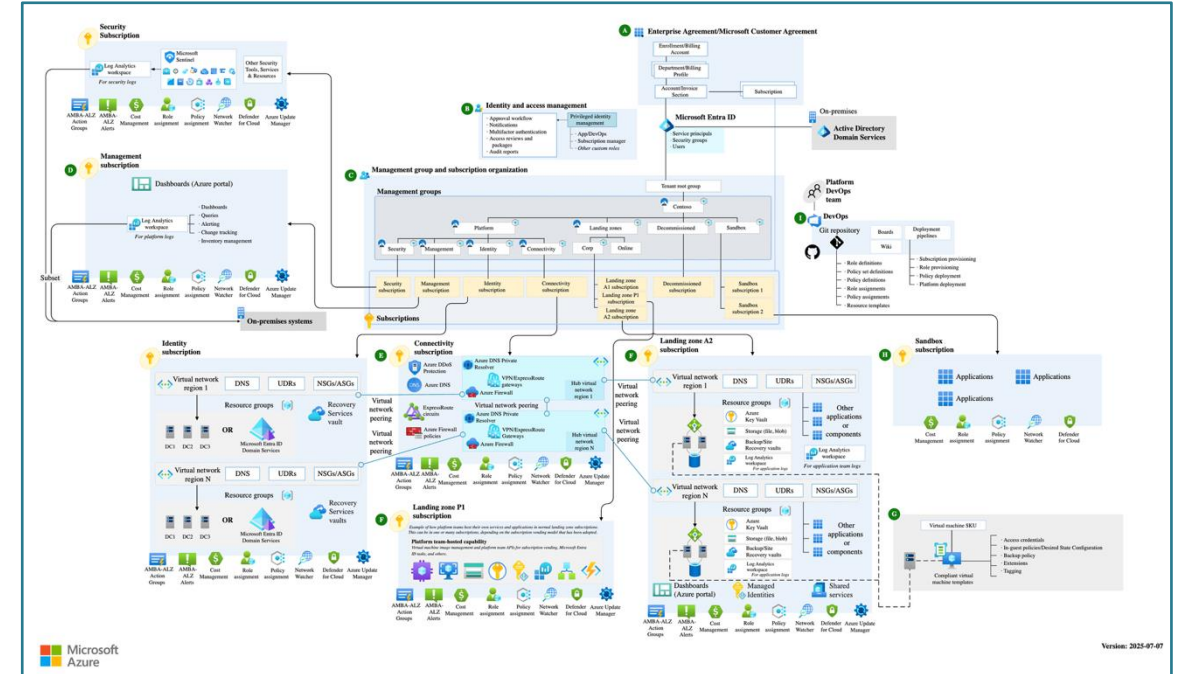
Resultat

- Landing Zone kann nicht einfach **ausgetauscht** werden
- Migration notwendig → Zeit- und Kostenintensiv

Theorie vs. Realität

Theorie

- Referenzarchitektur (CAF)
- Klare Governance
- Automatisierung



Realität (Typischer Projektverlauf)

- Workloads starten vor der Plattform → "Wir brauchen eine Subscription"
- ClickOps
- Teams arbeiten isoliert
- Wachstum → Probleme häufen sich

Theorie vs. Realität

Probleme

- Keine Trennung zwischen den Stages
- RBAC-Chaos (direkt Benutzer berechtigt, kein Konzept)
- Ressourcen Benamung nicht eindeutig
- Compliance Verstöße (Verschlüsselung, Region, etc.)
- keine reproduzierbaren Deployments
- Kostenüberwachung

Resultat

- Reorganisation → Plattform entsteht nachträglich

Governance entscheidet

Guardrails vor Deployments

- Policy Initiatives → Klein anfangen (Region, Deny Public Access, etc.)
- RBAC-Standardmodell
- Namenskonventionen forcieren (kein RegEx Support für Azure Policy)
- Default Tags (4-5) → Kostenstelle, Umgebung, Workload, Verantwortlicher
- Cost Management (Budgets)

Guardrails nachträglich einführen:

- Frustration in den Teams
- Deployments gehen kaputt
- Ausnahme über Ausnahme → Sinn von Governance?

Management Hierarchie - Zielbild

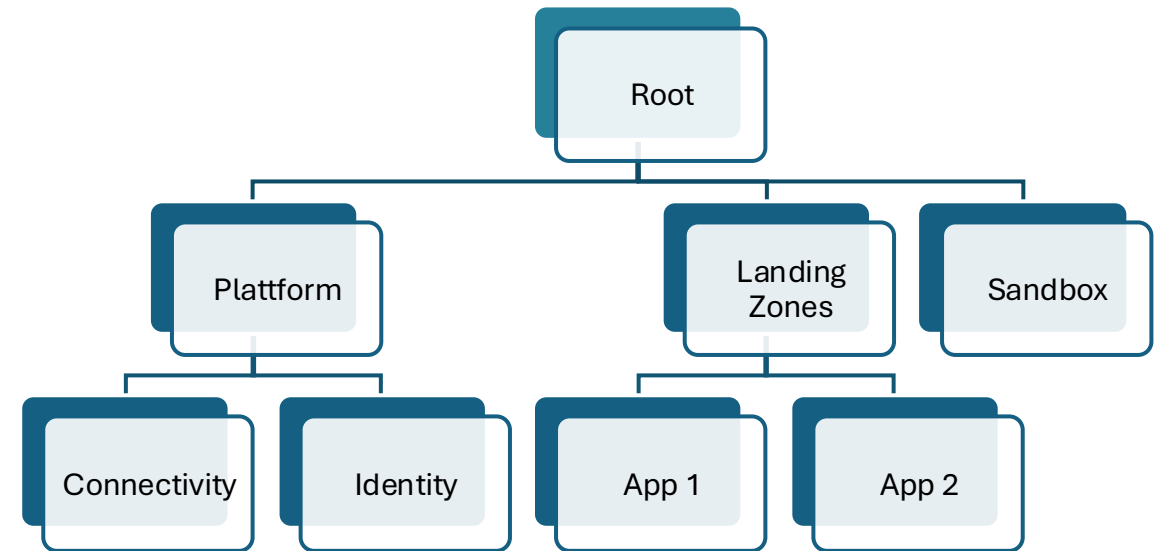
- Starke Orientierung am Microsoft Cloud Adoption Framework (CAF)
- Nicht nach nach **Stages** oder **Fachabteilungen** strukturieren

Warum CAF?

- Bewährtes Design („nicht das Rad neu erfinden“)
- Klare Trennung von **Plattform** / **Landing Zones** / **Sandbox**

Wenn die Struktur falsch gewählt ist:

- RBAC wird komplex
- Azure Policy Chaos
- Schlechte Skalierung
- Kosten-Reporting schwierig



Management Hierarchie – Anti-Pattern

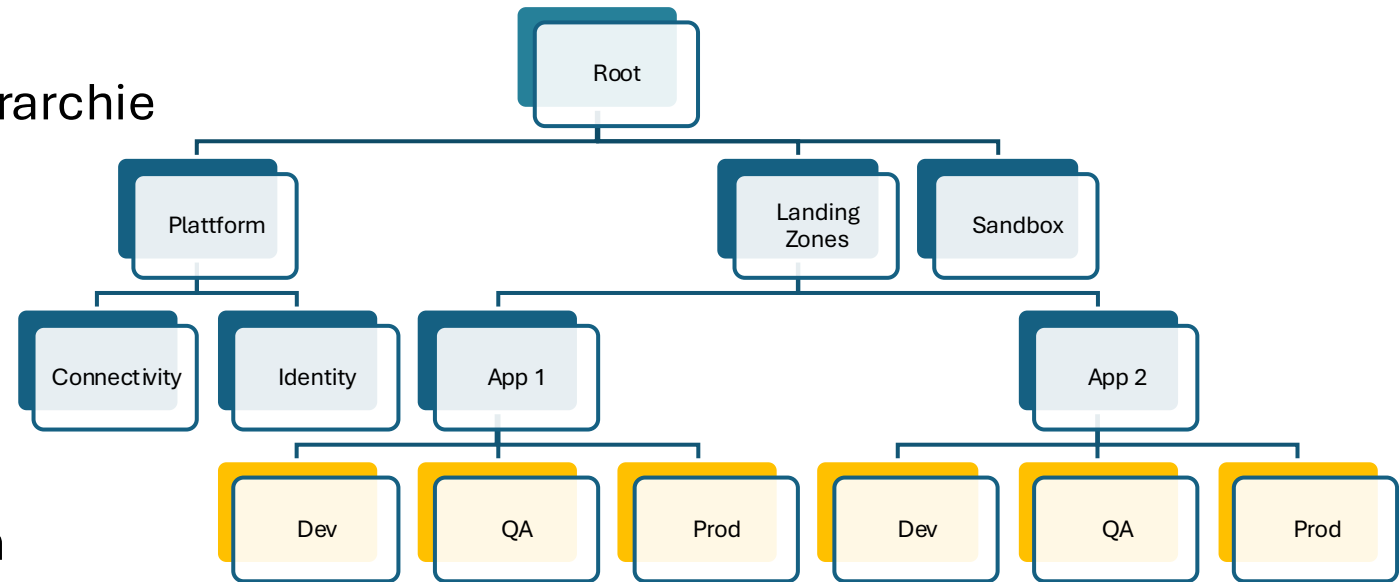
Staging innerhalb von Workloads

Technische Limits

- Max. 6 Ebenen in der Management Group Hierarchie
- Limitierte Anzahl an Management Groups

Warum Problematisch?

- Wirken logisch → Skalieren nicht!
- Explosion von Management Gruppen
- Policy Assignments per Stage unterschiedlich
- Deployment Fehler (Storage Account: Public Access)
- 30 Workloads = 120 Management Gruppen



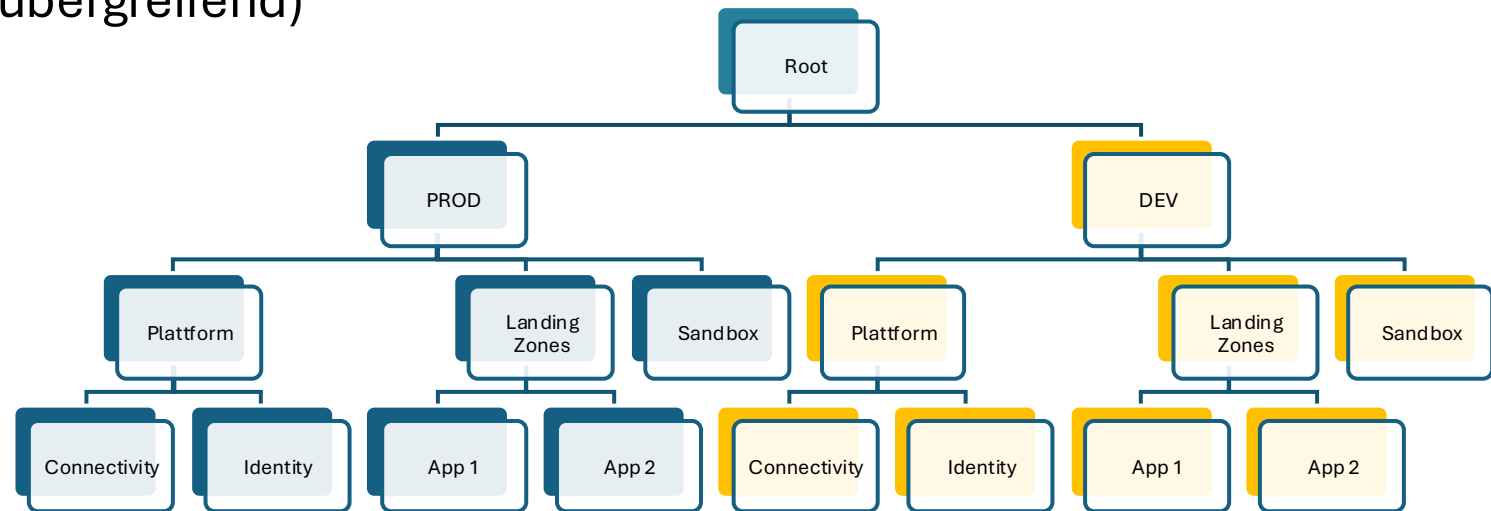
Management Hierarchie – Environment-based

Vorteile

- Oft Anforderung aus der Compliance
- Staging für Azure Policies (Workload-Ebene)

Nachteile

- Verwaltungsaufwand Policies per Environment
- Deployment Fehler (Stage-übergreifend)



Subscription Vending – Wieso wichtig?

 Bereitstellung erfolgt **automatisiert** nach Unternehmensstandard

Herausforderung bei manueller Erstellung

- Inkonsistente Zuordnung zu Management Gruppen
- Fehlendes Policy Enforcement
- Falsche/Fehlende RBAC-Zuweisungen
- Kostenkontrolle (Budgets fehlen)
- Plattform Team wird zum “**Bottleneck**”

Netzwerk - Keep it Simple, Stupid (KISS)

Funktioniert

- Hub-and-Spoke
- Azure Virtual WAN*
- Zentrale Connectivity
 - ExpressRoute & VPN Gateway
 - Azure Firewall & 3rd Party NVAs
 - Azure DNS Private Resolver
 - Private DNS Zones
 - Azure Bastion

Funktioniert nicht

- On-Premises Architektur kopieren → Cloud $\neq$ Datacenter
- zu viele Firewalls (Zonen) → Routing Hölle + Kosten
- Direktes Peering (inkl. Meshing)

Netzwerk - Azure Firewall oder NSG?



Better together 🧑🏫🧑🏻

- **Layered Security**

- NSG = Micro-Segmentierung (Subnet/NIC)
- Azure Firewall = zentralisierte Kontrolle und Inspektion

- **East-West vs. North-South**

- NSG kontrolliert internen VNet Traffic
- Firewall für Spoke-to-Spoke Kommunikation
- Firewall zuständig für Internet Traffic (UDR)

- **NSG für Basisfilterung**

- kein FQDN-Support
- L3/L4 Rules only

Netzwerk – Keep in mind

- **Troubleshooting dauert länger**
 - Firewall-/NSG-Logs sind nicht „mal eben“ geprüft
- **User-ID / Identity Filtering eingeschränkt**
 - Tech Preview läuft aktuell (AzFW)
- **SNAT Limitierungen (Static Rules)**
 - Keine feste Public IP pro Verbindung → Nutzung von **IP Pools**
- **Packet Capture eingeschränkt**
 - Limitierte Möglichkeiten, Storage Account und SAS URL notwendig
- **DNS ist kritisch !**
 - Essenziell für **Private Endpoints** und **Multi-Stage** Environments
- **Limitierungen berücksichtigen**
 - 500 Peerings pro Virtual Network
 - 50/90 Rule Collection Groups für Azure Firewall Policy

Netzwerk – „Shared Service Syndrom“

Definition

- Zentrale Services werden im Hub platziert
- Hub entwickelt sich zur **Compute-Zone** ⚠

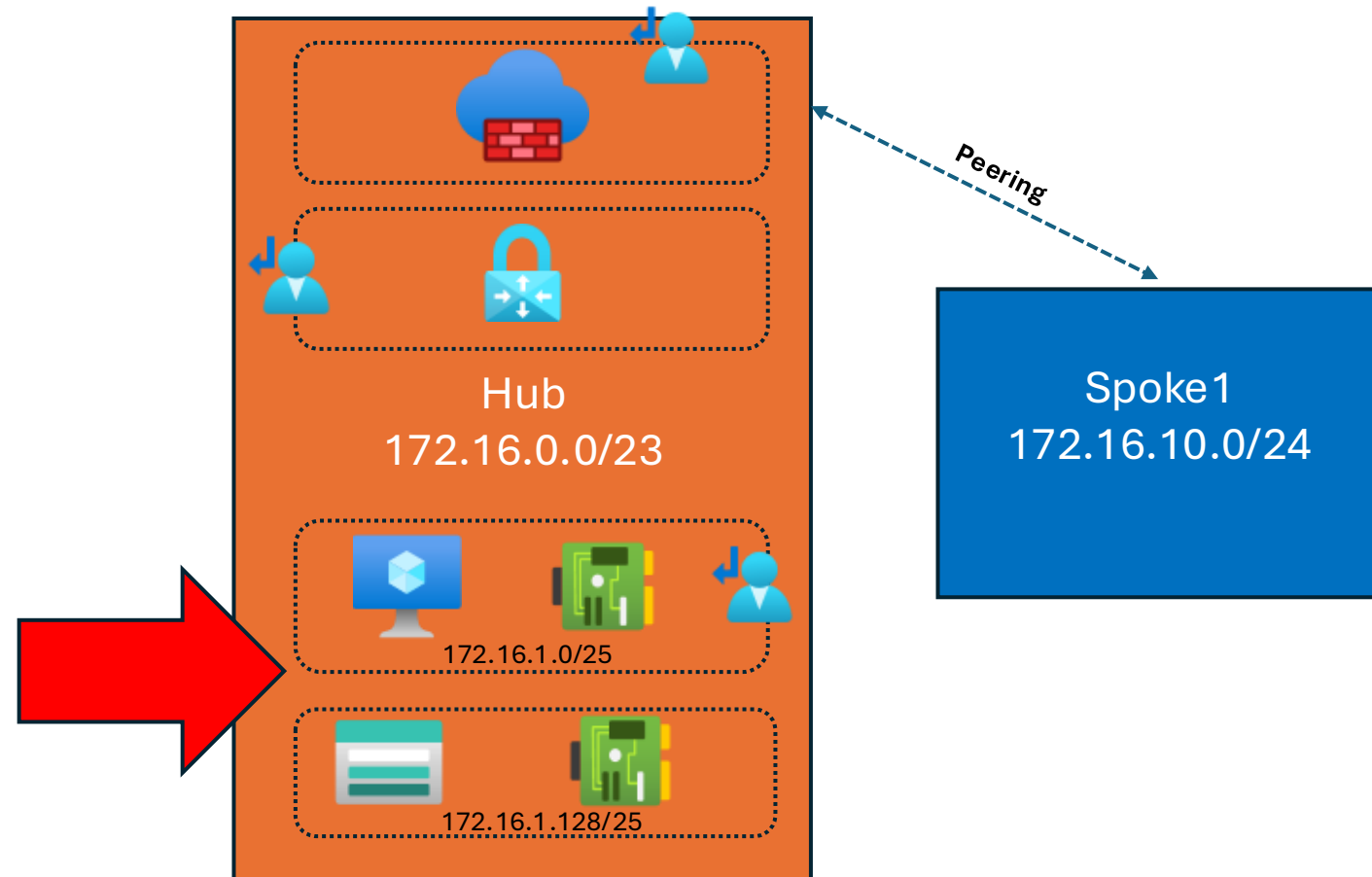
Typische Beispiele

- File Server im Hub
- Domain Controller im Hub
- Application Gateway im Hub

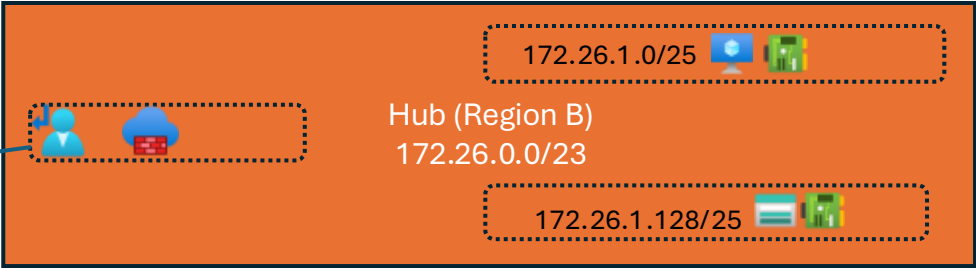
Probleme

- Security Bypass
- Komplexes Routing (UDRs)
- Asymetrisches Routing
- Skalierungsprobleme (Spokes & Multi-Region)

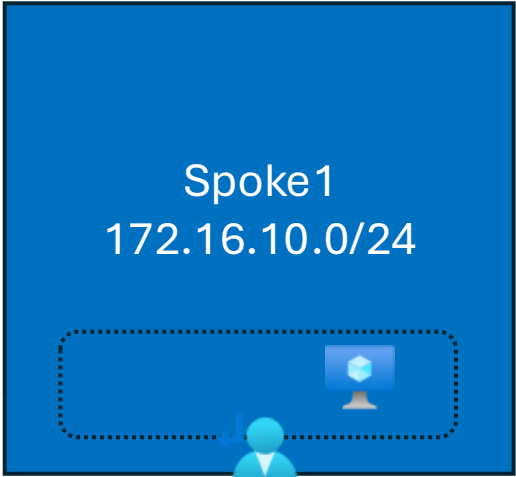
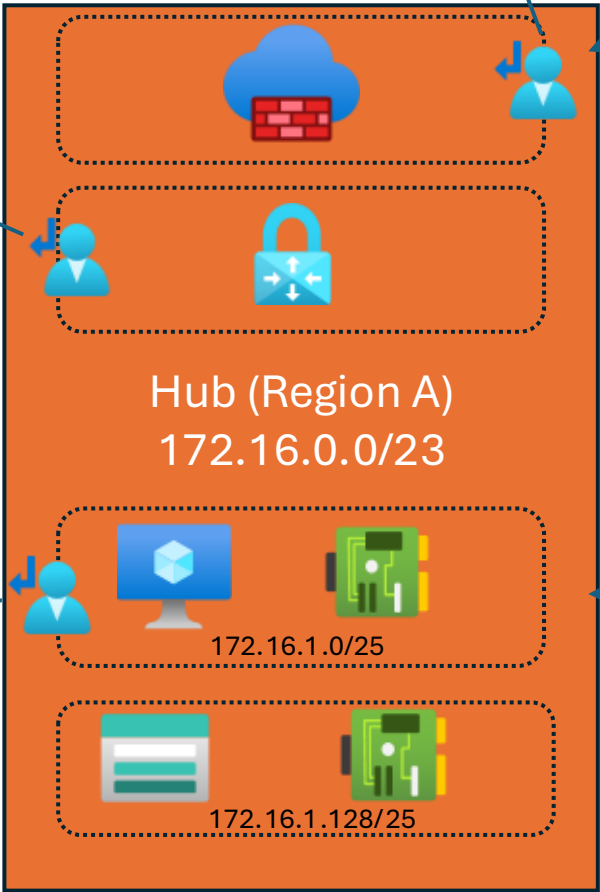
→ **Hub ≠ Shared Compute Layer**



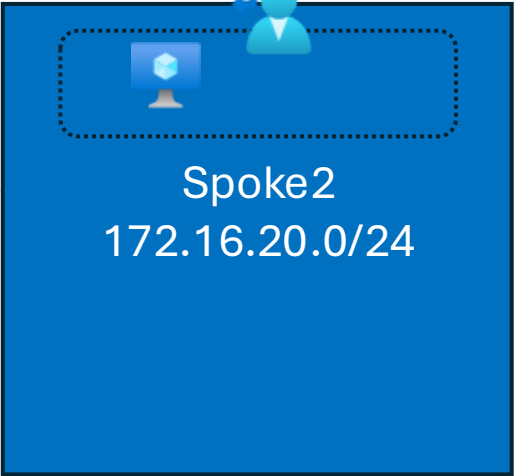
Routes	Next Hop
Region A Range (/16)	Azure Firewall Region A
Hub-Region1-SNET1 172.26.0.0/25	Azure Firewall Region A
Hub-Region1-SNET2 172.26.0.128/25	Azure Firewall Region A



Routes	Next Hop
Region B Range (/16)	Azure Firewall Region B
Hub-Region2-SNET1 172.26.1.0/25	Azure Firewall Region B
Hub-Region2-SNET2 172.26.1.128/25	Azure Firewall Region B



Routes	Next Hop
0.0.0.0/0	Azure Firewall
Hub-Compute-SNET-1 172.16.1.0/25	Azure Firewall
Hub-Compute-SNET-2 172.16.1.128/25	Azure Firewall



Routes	Next Hop
Spoke1-Compute-VNET 172.16.10.0/24	Azure Firewall
Spoke2-Compute-VNET 172.16.20.0/24	Azure Firewall
Hub-Compute-SNET-1 172.16.1.0/25	Azure Firewall
Hub-Compute-SNET-2 172.16.1.128/25	Azure Firewall

Routes	Next Hop
0.0.0.0/0	Azure Firewall
Spoke1-Compute-VNET 172.16.10.0/24	Azure Firewall
Spoke2-Compute-VNET 172.16.20.0/24	Azure Firewall
Hub-Compute-SNET-2 172.16.1.128/25	Azure Firewall

Peering

Peering

Peering

Netzwerk – „Shared Service Syndrom“

Zielbild

- Klare Trennung von **Connectivity** und **Workloads**
- Spoke-Default UDR: **0.0.0.0/0** auf Next-Hop: Azure Firewall ✓
- Gateway UDR: “Spoke” **CIDRs** auf Next-Hop: Azure Firewall ✓

Hub (Core Services)

- Azure Firewall / NVA
- VPN
- DNS
- Bastion

Spokes (Landing Zones)

- App-Services (Storage Account, VMs, etc.)

Was funktioniert gut

Muster erfolgreicher Implementierungen

- Klare Plattform-Ownership (nicht am Infrastruktur Team “vorbeiarbeiten”)
- An den Microsoft Best Practices orientieren → CAF = Guideline, kein geschriebenes Gesetz
- Automatisiertes Subscription-Vending
- Saubere Trennung von Umgebungen
- Governance von Anfang an
 - Azure Policy → Compliance/Security Abstimmung notwendig
 - Azure Hierarchy und RBAC-Konzept
- Infrastructure as Code
 - Auf gemeinsamen Tool-Stack einigen
 - Standardisierte Module und Templates (inkl. Linting, Testing)
 - Git Branching → Trunk-Based Development
 - Sauberer Pipeline-Release Prozess (CI/CD)

Warum Landing Zones scheitern

- Overengineering
 - Netzwerk Paranoia (On-Premises Mindset)
 - Tagging Wahnsinn (25+)
 - Hunderte von Custom Roles (RBAC) vs. Built-In Roles
 - Policy Perfektionismus → Fokus auf das Wesentliche!
- Referenzarchitektur blind kopiert
- Fehlende **Dokumentation**
- Infrastructure-as-Code ist schlecht umgesetzt
 - Hardocded Attribute (keine Wiederverwendbarkeit)
 - Module nicht anpassbar genug für unterschiedliche Workloads
 - Riesige Monolith-Deployments → Deployments dauern 60 Minuten+
 - Kein paralleles Arbeiten möglich (State-Lock)
 - Keine Trennung der Deployments nach Units (Network -> Baseline -> Storage -> Database)

Warum Landing Zones scheitern

- Zuständigkeiten müssen klar definiert sein
 - **Plattform:** IaC Standards, Netzwerk-Core, Subscription Vending, Self Services → Baut keine Workloads
 - **Security:** Defender for Cloud, Verschlüsselung, Compliance Baselines, SIEM)
 - **Governance:** Azure Policy, Naming und Tagging Standards, Budget Alerts)
 - **Workload (App/Dev Team):** Deployment und **Betrieb** ihrer Anwendungen
- Früh getroffene Designentscheidungen sind nicht skalierbar
- Eingeschränkte Flexibilität/Kostentransparenz durch “Shared Service” Mindset (AppGw)
- Konflikte zwischen den Teams (Starre Governance, fehlende Standardisierung)
- Kein Enablement für die notwendigen Technologien (**Lernkurve**)
 - Azure Know-How
 - Git (Source Control)
 - CI/CD
 - Terraform CLI
 - HashiCorp Cloud Plattform (HCP)

Was ist eine Landing Zone wirklich?

 Was eine Landing Zone NICHT ist

- Kein Projekt von ein paar Tagen oder Wochen
- Beschäftigung nur für das Netzwerk Team
- Terraform- oder Bicep-Repository
- Kein statisches Konstrukt

 Was eine Landing Zone IST

- Eine standardisierte Plattform
- Architektur + Governance + Prozesse
- Technisch UND organisatorisch
- Enabler für Teams → kein Bremse
- Ein Produkt, das kontinuierlich weiterentwickelt wird

Danke fürs
Zuhören!

Fragen?
Diskussion?

